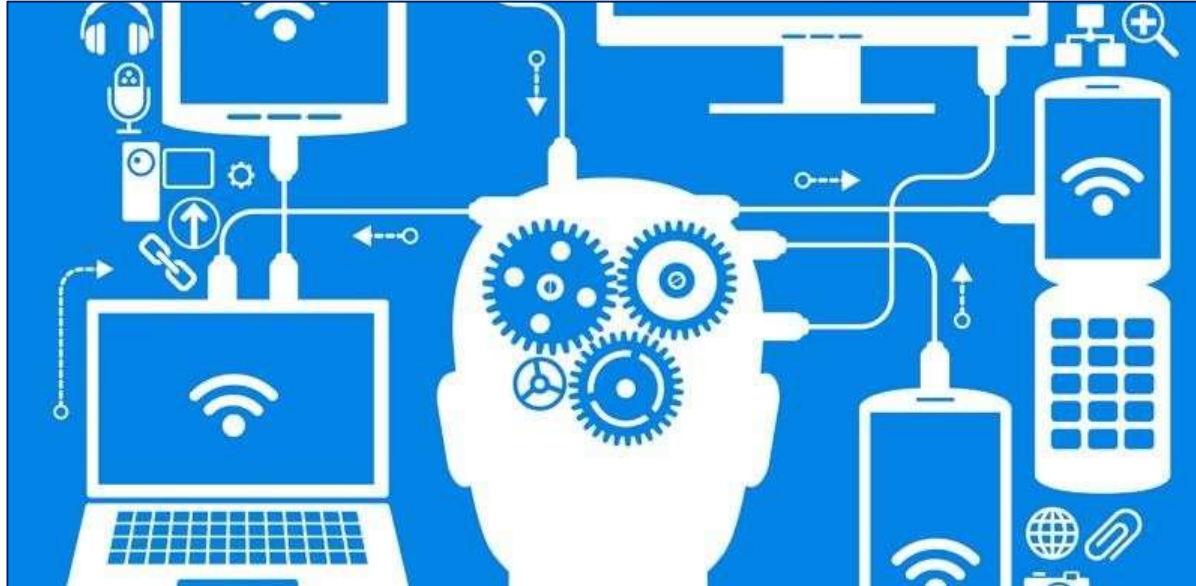


Software Engineering - 5CM505

Lecture 8 – UML # 2

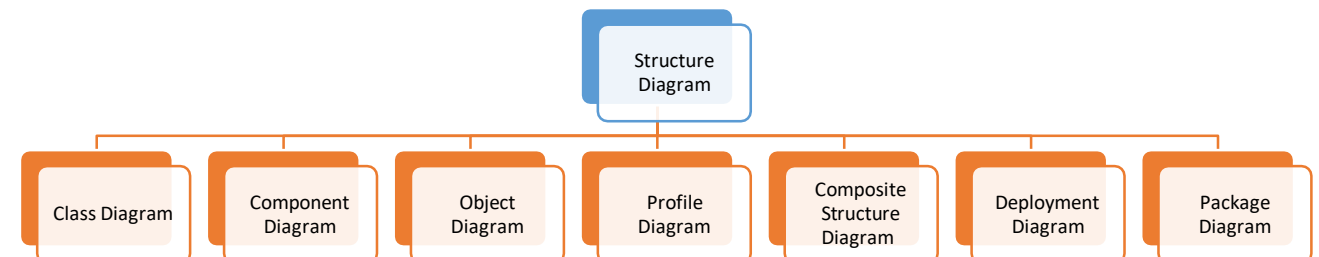
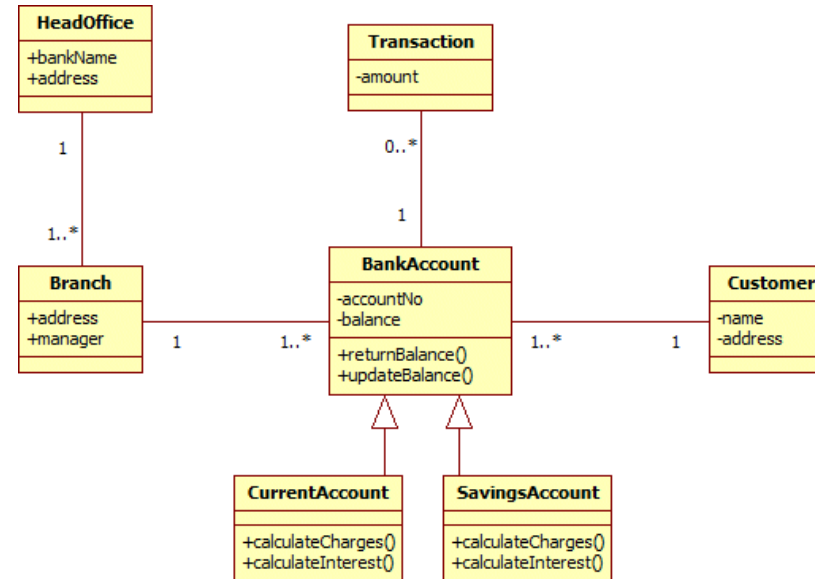
Ioannis Tsioulis (MC)



slides acknowledgements:
Dr Amanda Whitbrook (UoD)
Dr Saeed Ullah (UoD)

Lecture 8 - Content

- Recap from last week
- UML Structure Diagrams
 - Class Diagram
 - Component Diagram
 - Deployment Diagram
- UML Resources
- Summary



Learning Outcomes

- By the end of this lecture you should be able to:
 - Translate conceptual systems into the different kinds of UML diagrams we have looked at
 - Understand which diagrams are appropriate for modelling the different aspects or views of the system

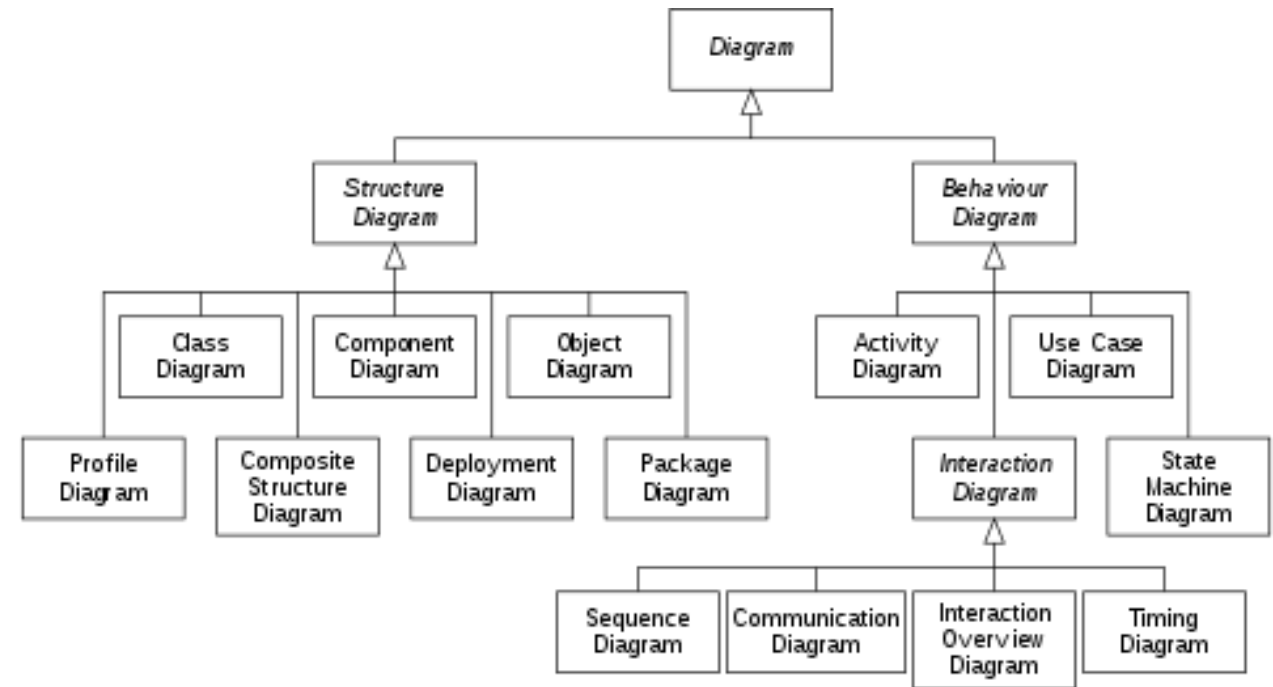


Learning Objectives

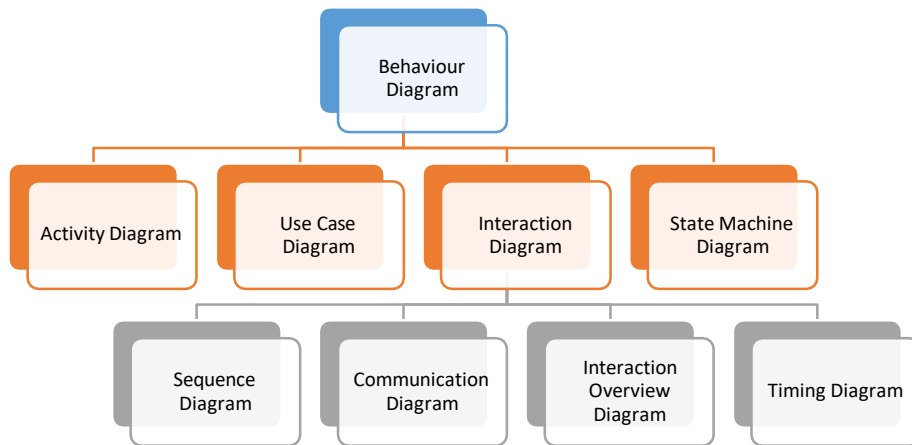
Recap from Last Week

UML Diagrams

- Several diagram types, but they fall into two distinct categories:
 - **Behaviour Diagrams** e.g. Use Case Diagram
 - **Structure Diagrams** e.g. Class Diagram
- What is the difference between the two categories?
- Why do we have different diagram types?



Behaviour Diagrams



- The focus is on what **must happen** in the system being modelled.
- The behaviour diagrams thus model **functionality or dynamics** of the system
- Interaction diagrams are a subset of behaviour diagrams:
- They emphasize the flow of control and data among the things in the system being modelled.

Last Week

- Last week we looked at four diagrams in the Behaviour Diagrams category:
 - Use Case Diagrams
 - State Machine Diagrams
 - Activity Diagrams
 - Sequence Diagram
 - Communication Diagram



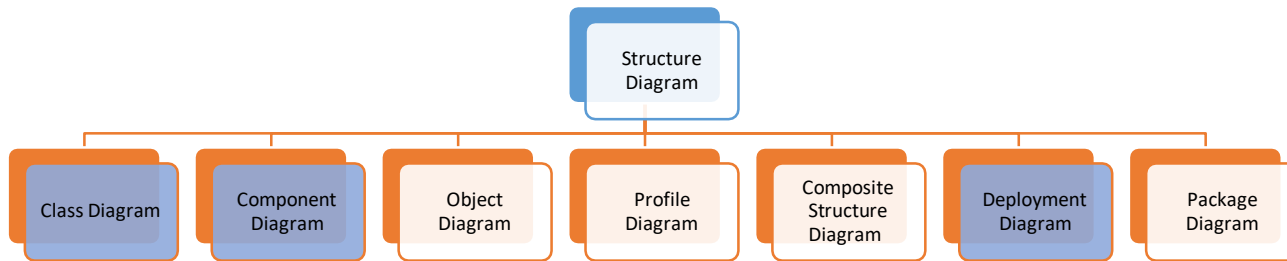
- What is a Use Case Diagram?
- What is it particularly useful for?

- What is a State Machine Diagram?
- What does it show that the Use Case Diagram does not?

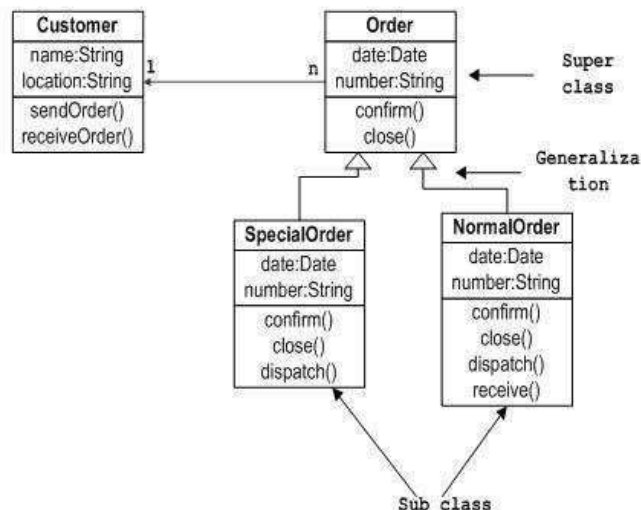
- What is an Activity Diagram?
- How does it differ from a State Machine Diagram?

UML Structure Diagrams

Structure Diagrams



Sample
Class
Diagram

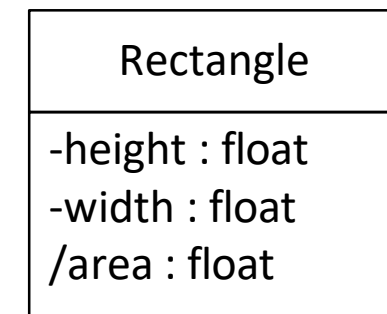
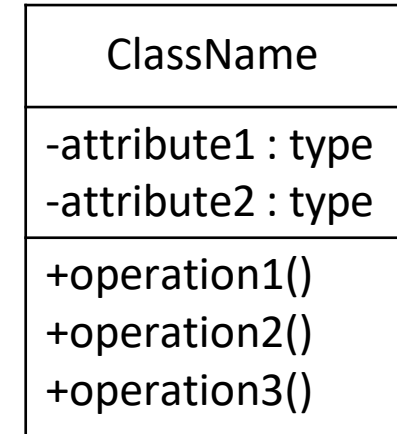


- The focus is on **what is present** in the system being modelled - what has to be there?
- The structure diagrams are thus concerned with modelling the **system architecture**
- The diagrams view the system from a **static** perspective

Class Diagram

Class Diagram

- **Class Diagrams** show the **classes** that exist in a system, and the **relationships** between them
- Each class is shown as a **rectangular box** and can be optionally divided into **three compartments**:
 - The **class name** is shown in the top compartment
 - The middle compartment shows the **attributes** of the class. Each attribute has a name, and other optional information may also be shown, e.g. type, initial value, visibility (public, private, protected ...).
 - The lower compartment shows the **operations** of the class. An operation has a name, and, optionally, a return type, parameter list and a visibility (public, private, protected ...). Some operations do not need to be modelled as they are automatically implied by most UML tools e.g. Get() and Set(). The notation <<constructor>> can be used to indicate constructor operations.
- **Derived attributes** can be shown with a /
- **Static attributes** (value is the same across all instances) and **operations** are shown by underlining them.

Base
attributesDerived
attribute

Visibility

- You can define what is **visible to other classes** and what is not by preceding the feature with a symbol to represent **Access Modifiers**

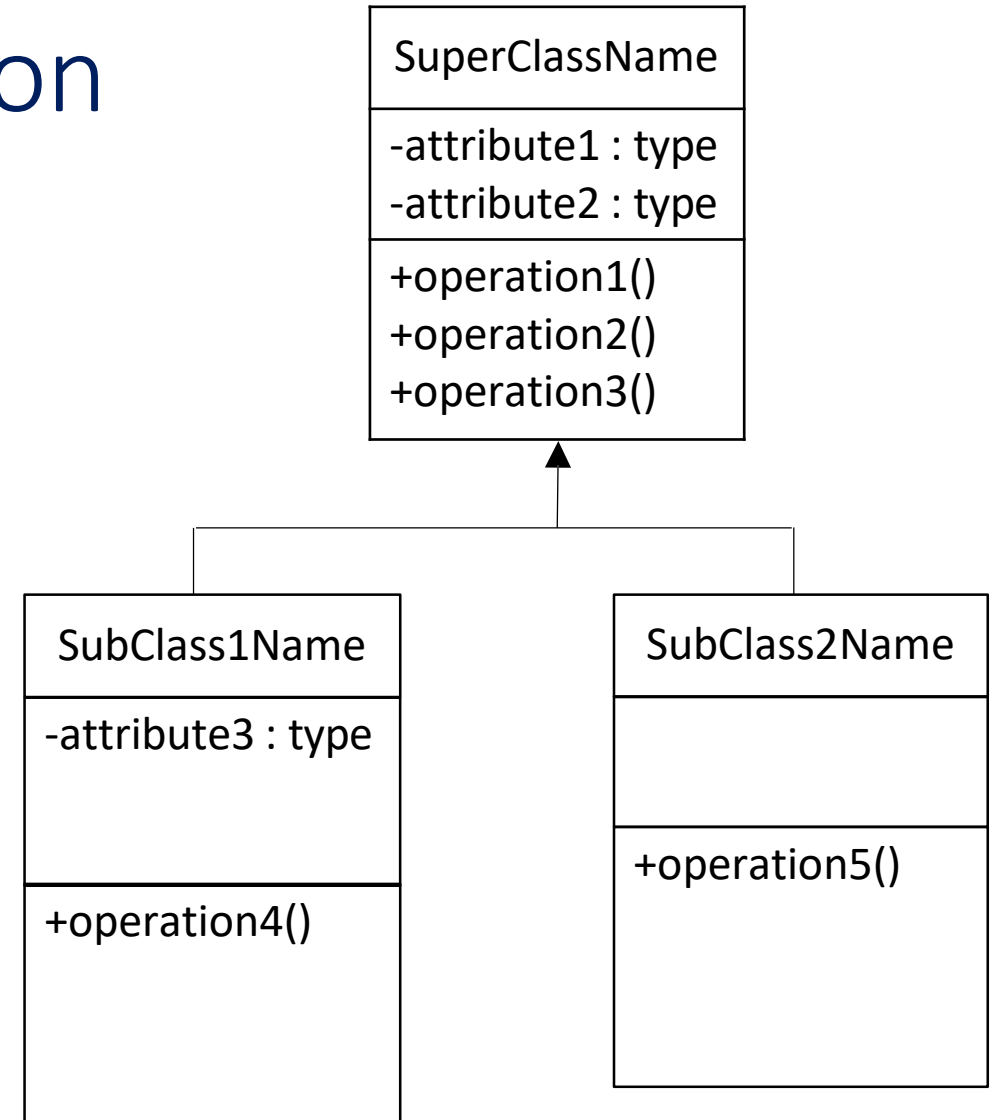
- Symbols used:

○ Private	–	Only the owning class can use the feature
○ Public	+	Any class can use the feature
○ Protected	#	Only the owning class or descendants of it can use the feature
○ Package	~	Only the objects in the same package as the owning class can use the feature

ClassName
-attribute1 : type
-attribute2 : type
+operation1()
+operation2()
+operation3()

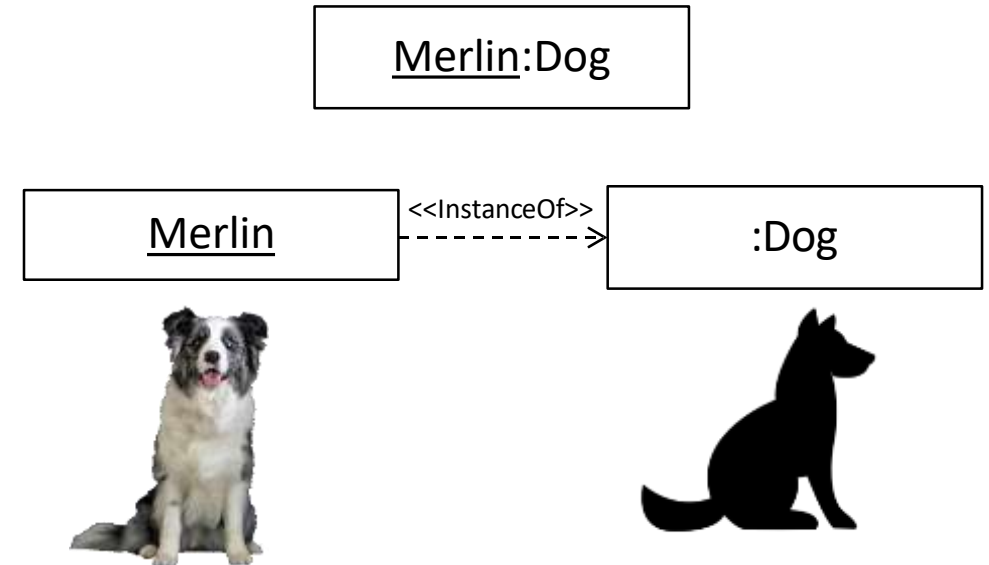
Class Diagram Generalization

- Classes can be related by **generalization**
- A generalization **relates** one **superclass** to one or more **sub-classes**
- Attributes and operations of the superclass are **inherited** by the subclass
- The subclass can also add **new features** not present in the superclass and can **redefine superclass features** if necessary

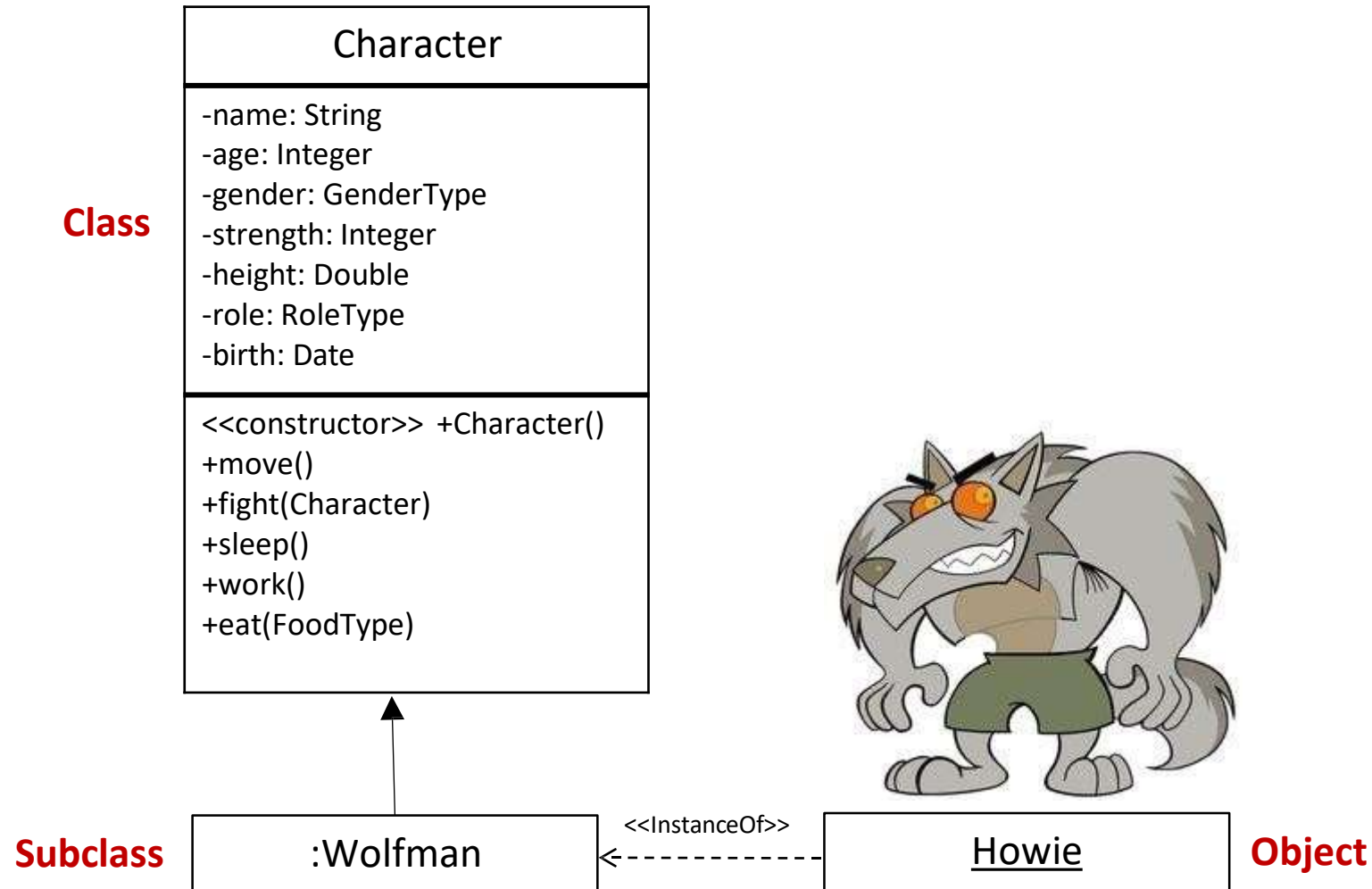


Class Diagram Objects

- **Objects** are also displayed in class diagrams using a rectangular box
- To distinguish them from classes either
 1. The name is underlined and followed by the class name with the two names separated by a colon, or
 2. A dashed arrow can be drawn from the object to the class, although this is used less often. Optionally, you can also label the arrow using <<InstanceOf>>

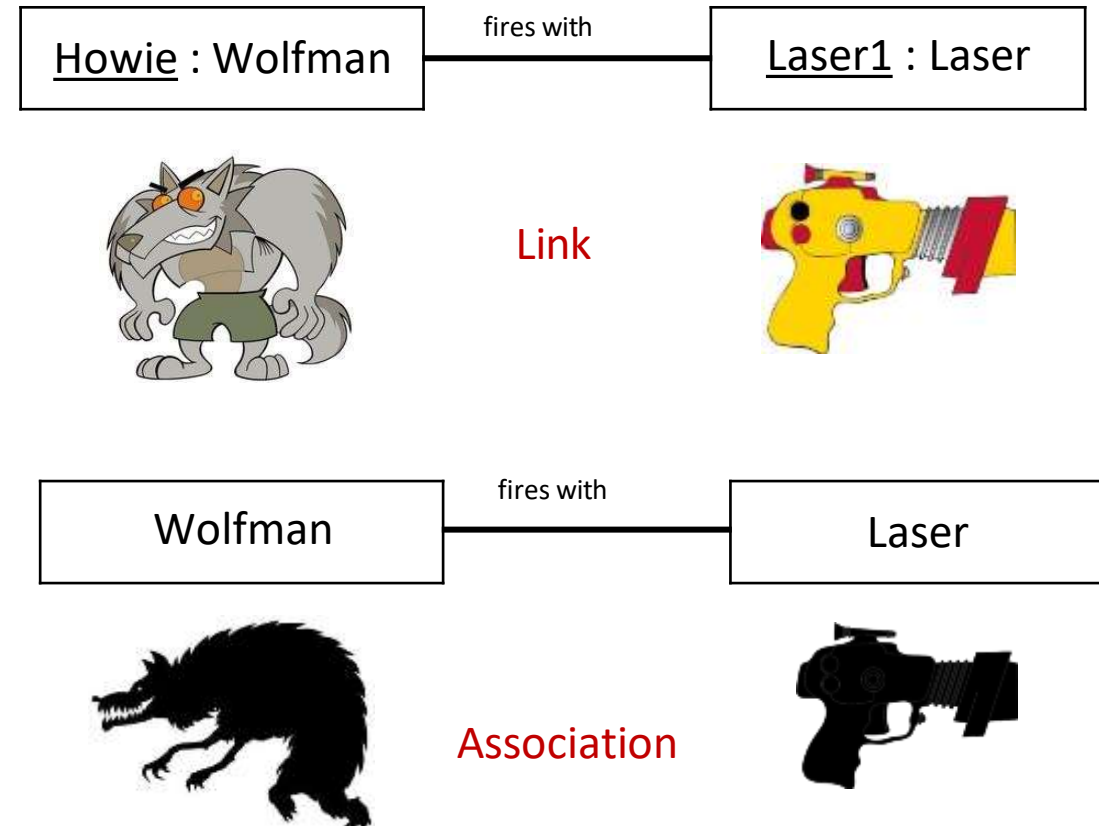


Class and Object Example: Video Game Characters



Links and Associations

- A **link** relates two **objects**. A link is represented by a line connecting the two boxes. An optional name for the link may also be shown above the line.
- The link should read left to right or top to bottom. If this is not possible (e.g. If the diagram is too big then use an arrowhead)
- In the example, the link means that Howie can invoke Laser1's operations.
- An **association** relates two **classes**, and is represented in the same way as a link.

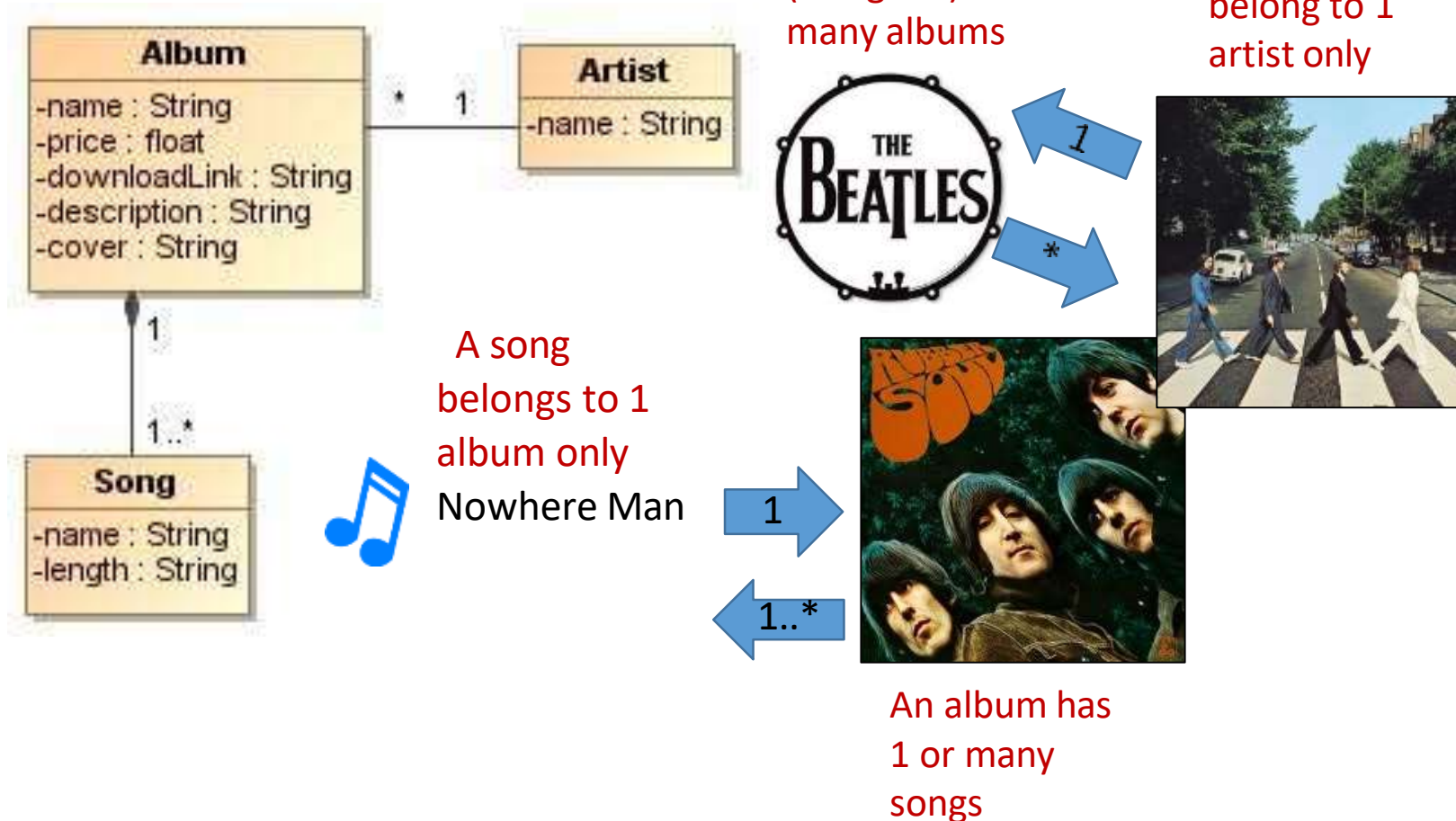


Multiplicity

- Symbols can be used to show **multiplicity** in associations and links, e.g. If you want to link a class to many instances of another class
- Multiplicity numbers are shown above or below the association line on each end

Symbol	Meaning
1	The class instance is linked with exactly one instance of the other class
* OR 0..*	The class instance may be linked with zero or more instances of the other class
0..1	The class instance may be linked with zero or one instance of the other class
1..*	The class instance must be linked with at least one or more instances of the other class
x..y	The class instance may be linked to at least x instances of the other class but not more than y
x,y,z	The class instance may be linked to x, y or z instances of the other class

Multiplicity Example



Group Exercise

- Complete the multiplicity labels for the associations on the diagram

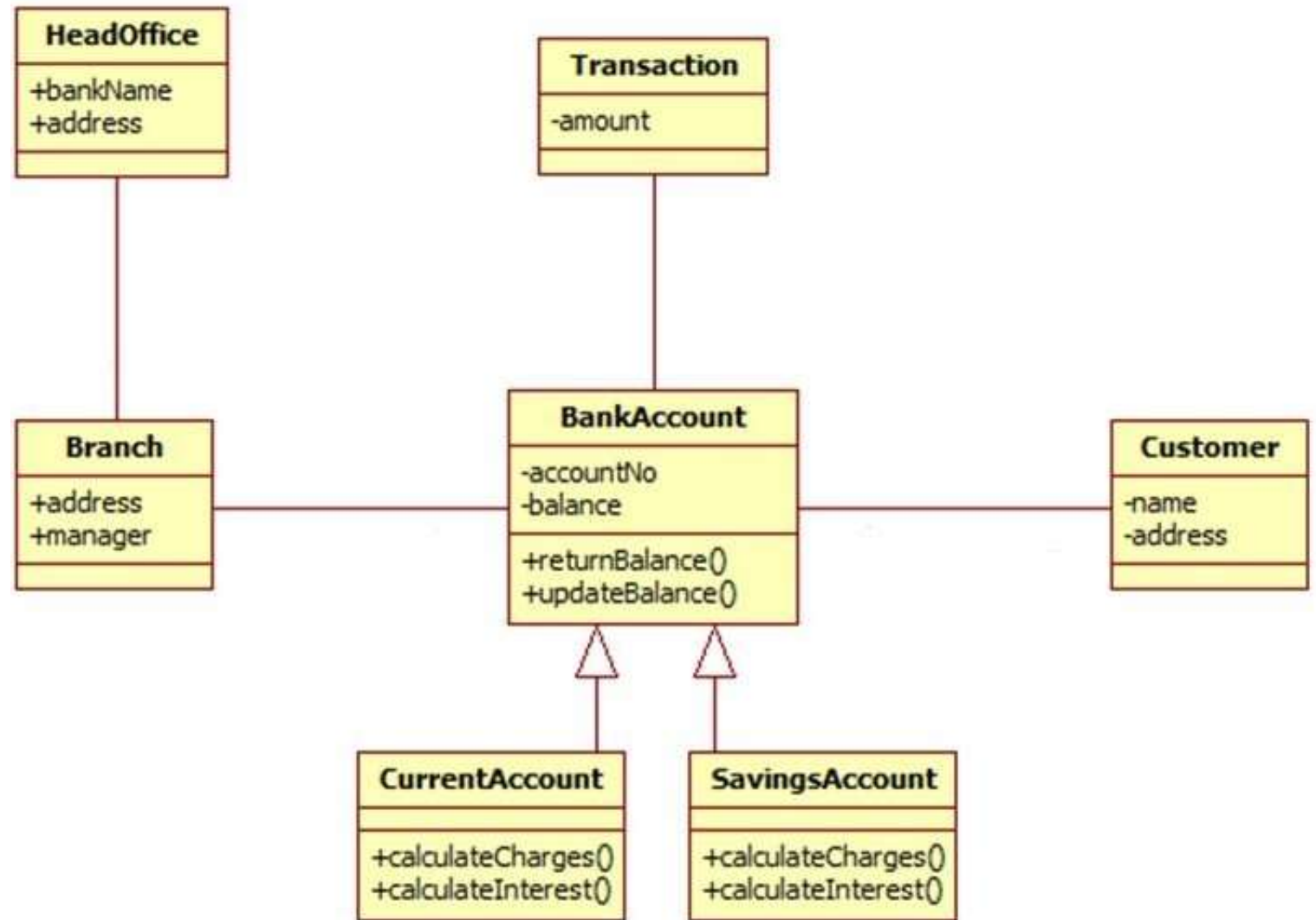
(5 mins)





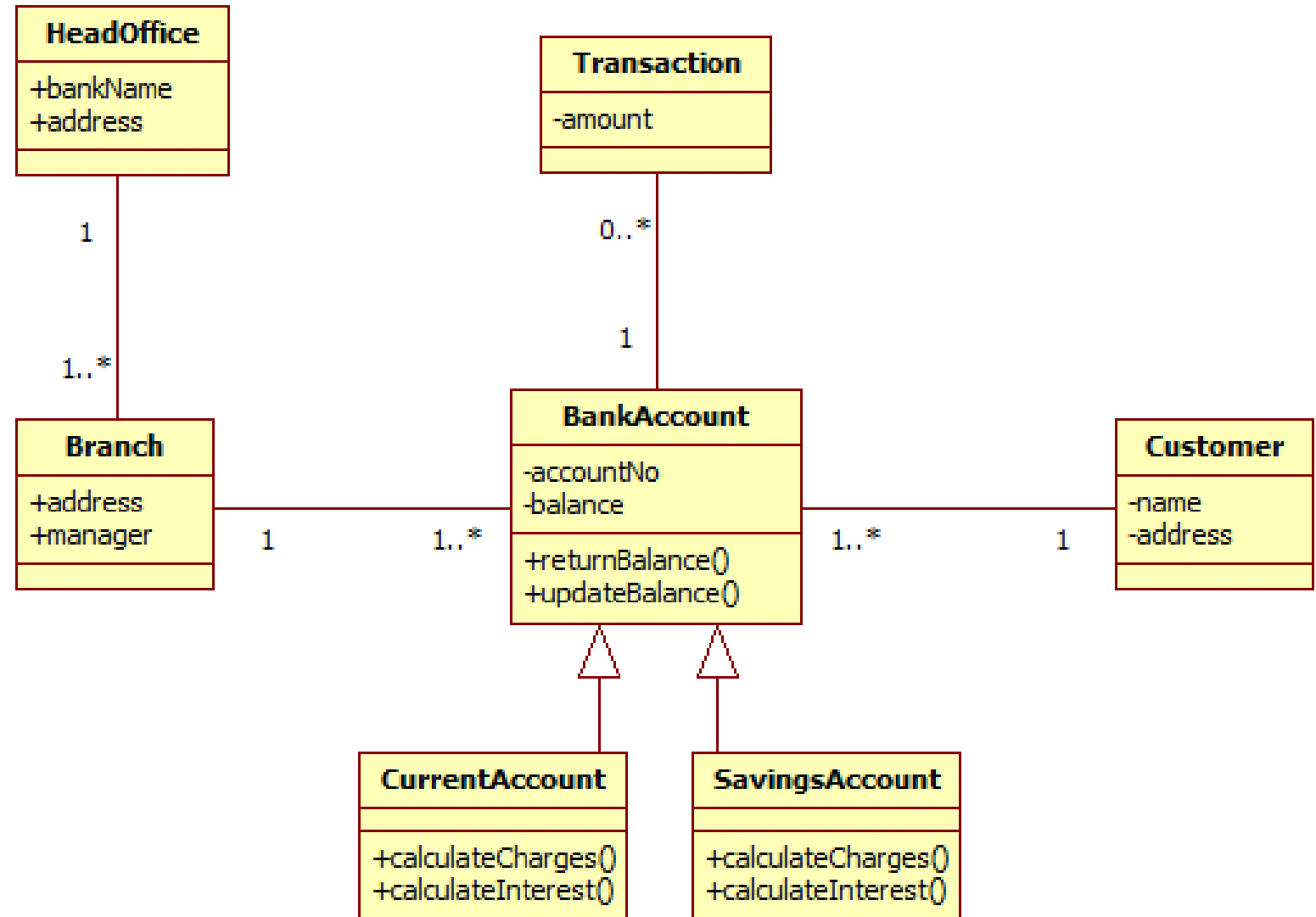
Banking Class Diagram

Class Diagram Exercise





Banking Class Diagram



Class Diagram Exercise 1

Component Diagram

Component Diagram

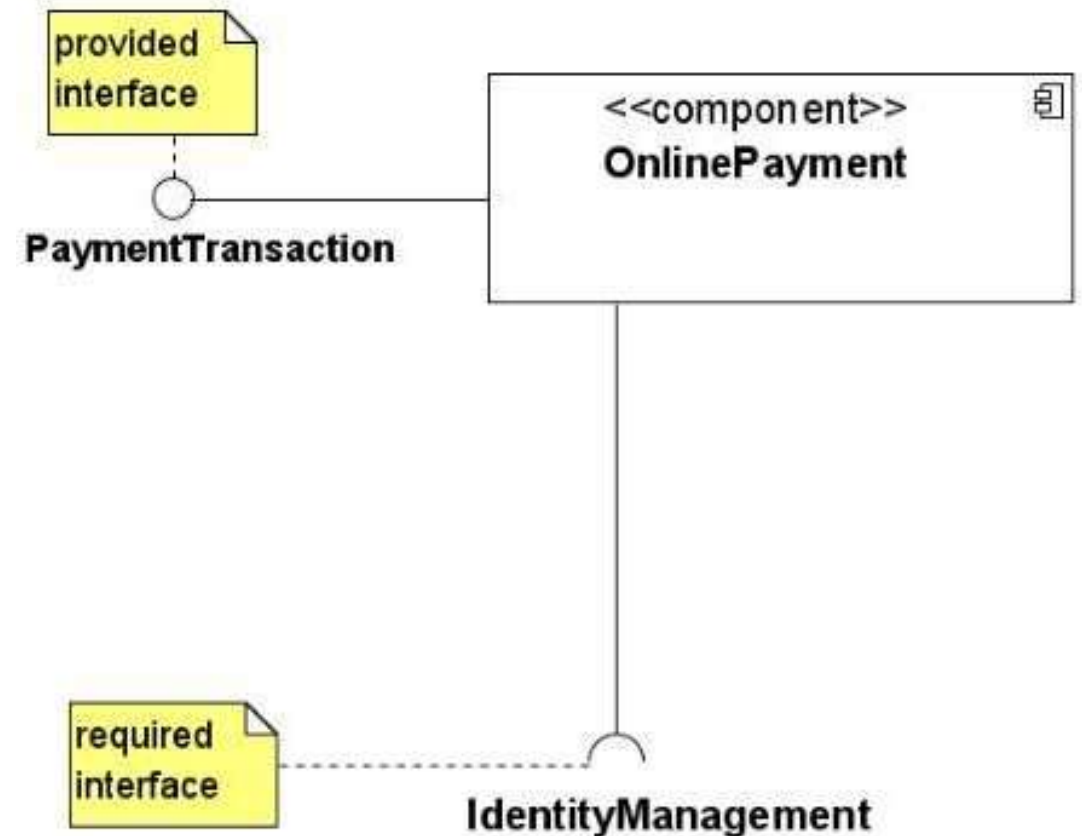
- **Components** are defined as physical, deployable and replaceable parts of a system. For example, a compiled object file, a piece of source code or a shared library.
- The **Component Diagram** shows how the components connect to make up a system.
- Components are represented by a rectangle with the name of the component inside. Optionally the name may be preceded by <<component>>. There may also be a small box with two tabs inside the rectangle



- Components encapsulate implementation details and expose certain interfaces to other parts of the system.
- They are depicted as “black boxes”. Which means that the specification detail is hidden – the focus is on the relationship between components

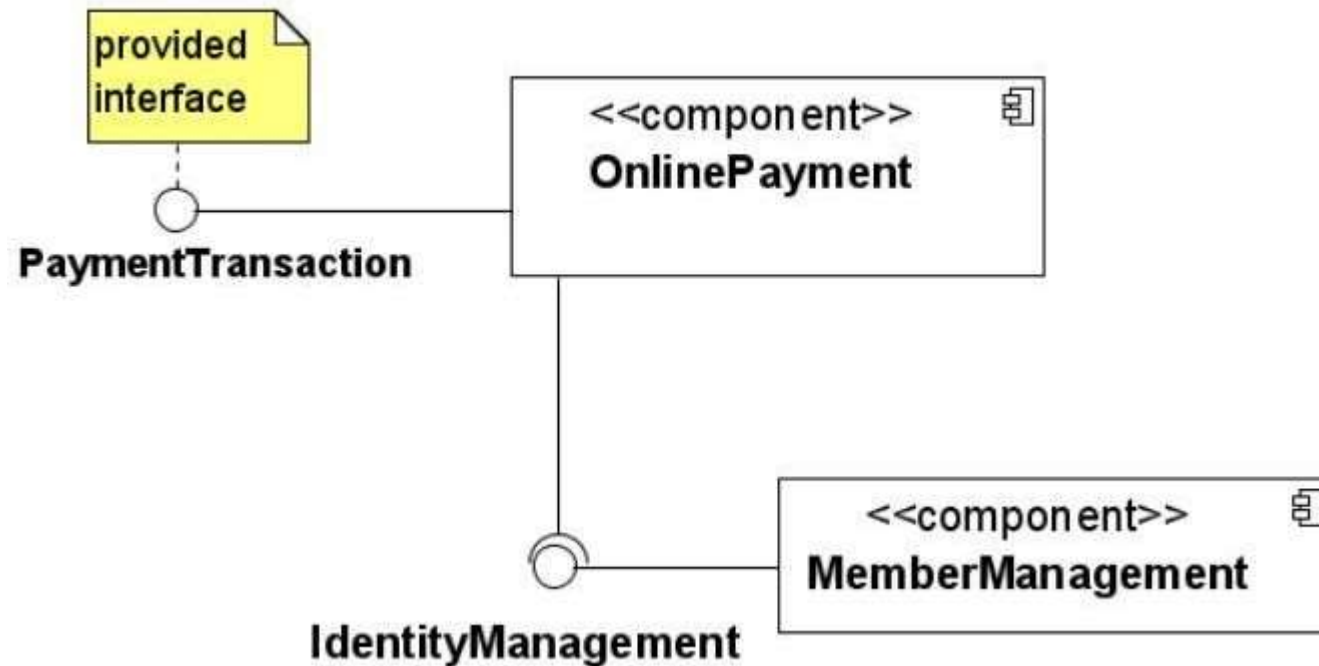
Component Diagram

- Each interface that a component provides to the world looks like a lollipop (a circle on a stick)
- Each interface that the component requires (from some other component in the system) looks like a half-circle on a stick

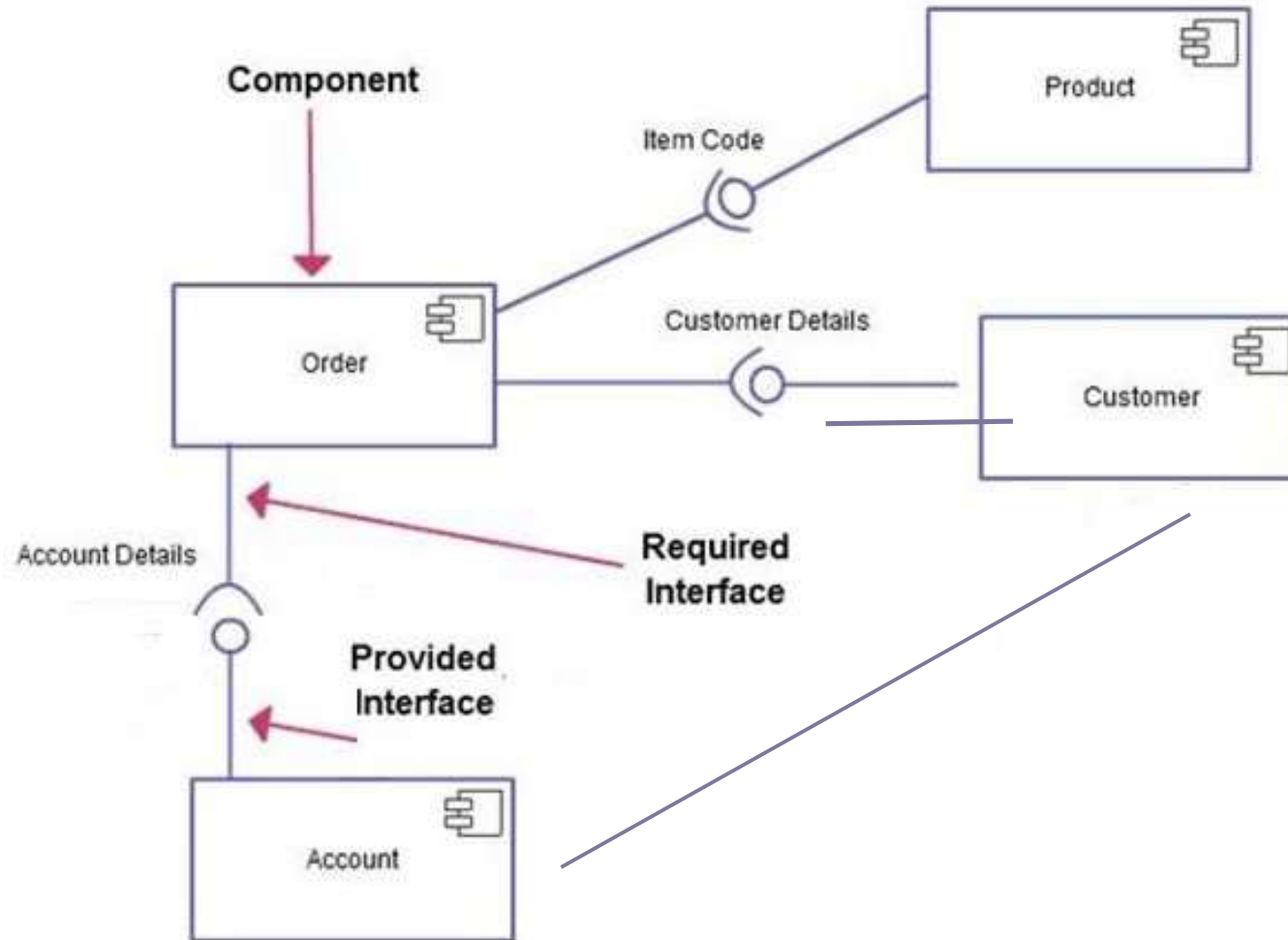


Component Diagram

Components can
be wired together
to form subsystems



Component Diagram Example



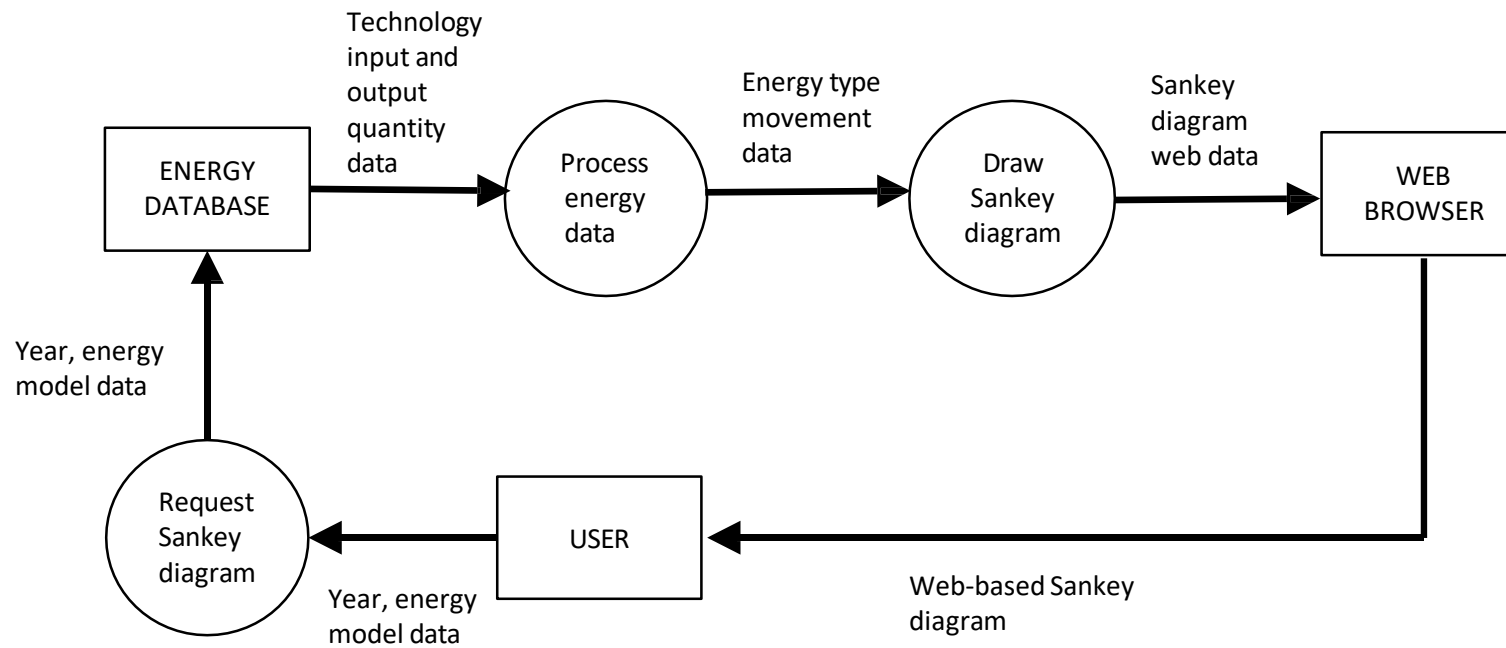
Group Exercise

Turn the FFD (Function Flow Diagram) for the Sankey Diagram Generator into a Component Diagram

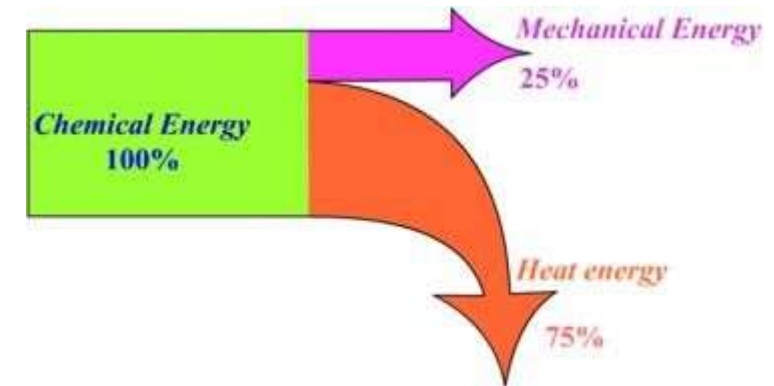
(10 minutes)



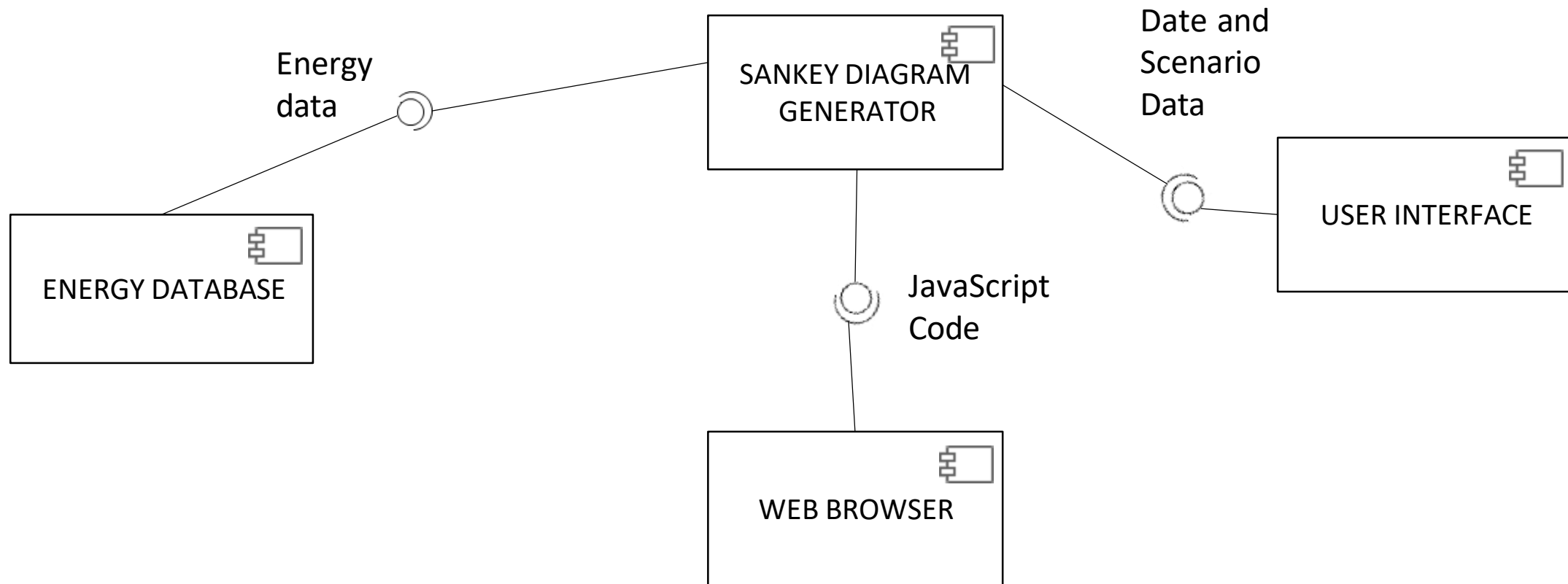
Sankey Diagram Generator – FDD Diagram



Example
2020, No nuclear



Sankey Diagram Generator – Component Diagram



Deployment Diagram

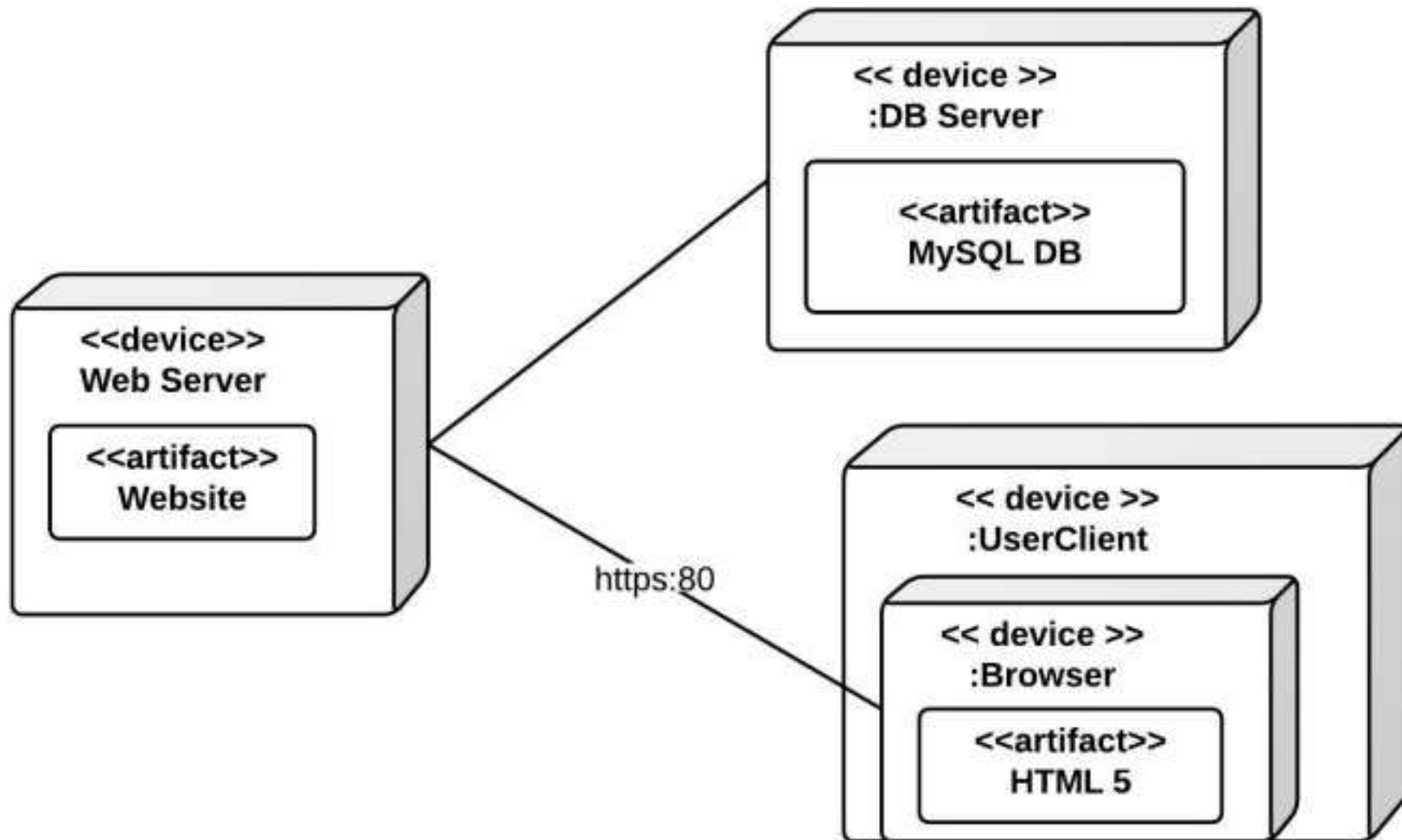
Deployment Diagram

- A **deployment** diagram models the **physical deployment** of **software components** (artefacts) on **hardware components** (devices).
- Example
 - To describe a client computer viewing a web site, a deployment diagram might show the following hardware components (devices)
 - A web server, a client computer, a database server
 - and the following software components (artefacts)
 - A web site, MySQL database software, HTML5

Deployment Diagram

- There are two types of node:
 - **Device node**
 - **Execution Environment node**
- **Device nodes** are physical computing resources with processing memory and services to execute software, such as typical computers or mobile phones.
- An **Execution Environment node** (EEN) is a software computing resource that runs within an outer node and which itself provides a service to host and execute other executable software elements.

Deployment Diagram Example



What do you notice about this diagram that does not really make sense?

Deployment Diagram

- The browser software on the user client is referred to as a device!
- This is because there are two types of hardware component:
 - Actual devices
 - Execution environment “devices”
- Actual devices are physical computing resources with processing memory and services to execute software, such as typical computers or mobile phones.
- An execution environment “device” is a **software** computing resource that runs within an outer node and which itself **provides a service to host and execute other executable software elements**.
- This makes deployment diagrams rather quirky!

Resources

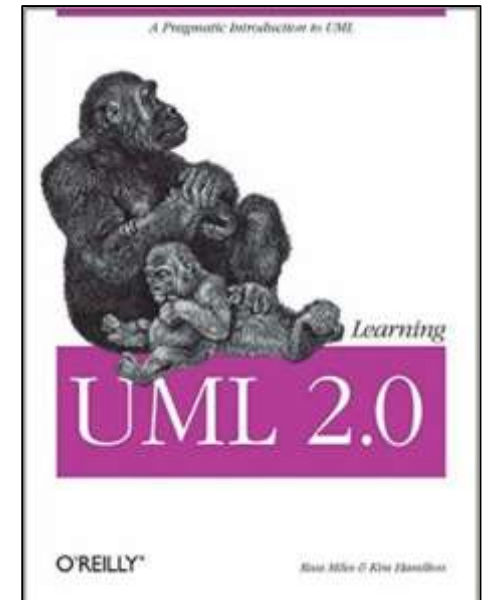
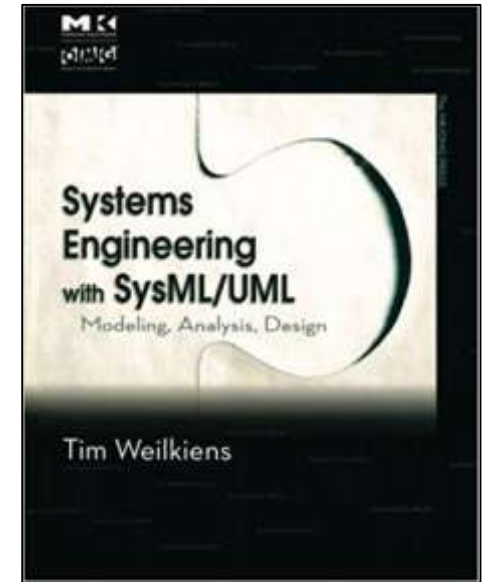
UML Resources

○ Web Links

- <http://www.uml.org/what-is-uml.htm>
- <http://www.uml.org/resource-hub.htm>
- http://www.sparxsystems.com.au/platforms/uml_resources.html
- <http://www.omg.org/>
- https://www.tutorialspoint.com/uml/uml_useful_resources.htm
- https://en.wikipedia.org/wiki/Unified_Modeling_Language (Good tutorial on UML)

○ Books

- Weilkiens, T. (2008). Systems Engineering with SysML/UML: Modeling, Analysis, Design. Morgan Kaufmann.
- Miles, R. & Hamilton, K. (2006). Learning UML 2.0. O'Reilly Media



Hints for Using UML

Hints for Using UML

- UML is a tool to help produce software. It provides a common language to communicate about a system without source code. **Producing UML is rarely a goal in itself.**
- In Agile development, UML diagrams should be produced only as needed to communicate effectively. **Their use should expedite development by facilitating group understanding, not slow it down.**
- Although there are numerous tools to help generate UML diagrams, unless the goal of development is to produce UML diagrams (see above), such tools are unnecessary.
- The best tools for producing and using UML diagrams are:
 - Pencils & eraser, plus paper
 - Whiteboard



Summary

- The last two lectures should have enabled you to start translating conceptual systems into the different kinds of UML diagram we have looked at.
- However, we have only really scratched the surface with some fairly simple examples. If you want to explore further, then I recommend getting hold of the books and exploring the other resources
- Remember that we have not covered all of the diagrams. If you want to include a UML diagram type, we have not looked at in your assignment portfolio, then please feel free to do so.
- NOTE
 - When submitting a UML diagram as part of your portfolio it is a good idea to explain why you used it. What information was it helping to communicate between team members? Did it help with the design or coding process?



Thank You!

